



Transactions

Isolation

Isolation is a property that defines how and when concurrent operations performed by others are made visible to a transaction.

Most DBMS offer certain levels of isolation that control the degree of access to data and try to avoid three unwanted phenomena in concurrence: dirty read, fuzzy read, and phantom row.

The ANSI/ISO SQL standard defines isolation levels:

- Read Uncommitted
- Read Committed (Write Lock)
- Repeatable Read (Read and write blocks)
- Serializable (higher isolation level; Read, write, and range locks)

The appropriate level of isolation for each application depends on the data integrity requirements of the application and the overhead it introduces from each isolation level.

Concurrency control

To ensure the level of isolation, DBMS use locks (Shared Lock S-lock or Exclusive Lock X-Lock) or multi-versioning concurrency control (MVCC). The following is the general behavior; Different DBMS can implement modifications.

- **Pessimistic approach or Block-based**

READ UNCOMMITTED (RU)

Operation	Lock	It is established	If free
SELECT	None	Not applicable	Not applicable
UPDATE/DELETE/INSERT	X-lock	Modifying / inserting row	Commit / Rollback

READ COMMITTED (RC)

Operation	Lock	It is established	Free
SELECT	S-lock over rows read	When reading each row	After reading the row
UPDATE/DELETE/INSERT	X-lock on row add/del/upd	When modifying/inserting row	Commit / Rollback

REPEATABLE READ (RR)

Operation	Lock	It is established	Free
SELECT	S-lock on read rows	By reading each row that meets the	Commit / Rollback
UPDATE/DELETE/INSERT	X-lock on row add/del/upd	When modifying/inserting row	Commit / Rollback

SERIALIZABLE (Ser)

Operation	Lock	It is established	Free
SELECT	S-lock on read rows and range if query acts on ranges	When reading rows or ranges	Commit / Rollback
UPDATE/DELETE/INSERT	X-lock	When modifying rows/ranges	Commit / Rollback



- **Optimistic approach or Multi-Version Concurrency Control**

READ UNCOMMITTED (RU)

MVCC cannot operate in READ UNCOMMITTED mode.

READ COMMITTED (RC)

When you get the version of the data	What version of data you see	Commit validations
At the beginning of each SQL statement	Versions committed before the read operation	Other transactions did not commit updates or deletes on rows that this transaction updates (avoids lost-update)

REPEATABLE READ (RR)

When you get the version of the data	What version of data you see	Commit validations
At the beginning of the transaction	Versions committed before the start of the transaction	Idem RC; No rows read during the txn have been modified or deleted by others since the start of the txn

SERIALIZABLE (Ser)

When you get the version of the data	What version of data you see	Commit validations
At the beginning of the transaction	Versions committed before the start of the transaction	Idem RR; No new rows have appeared that should have been visible for any queries during the txn

Oracle and HSQLDB

The two DBMS that we are going to use in this practice have the following characteristics in terms of concurrency control:

By default, they both work in **autocommit**, i.e. each sql statement is a transaction on its own and is automatically committed if it ends well. Therefore, in both cases you will have to disable Autocommit mode in both.

- **Oracle**

Implement optimistic concurrency control (MVCC). Supports READ COMMITTED and SERIALIZABLE isolation levels as defined in the standard (https://docs.oracle.com/cd/B10501_01/server.920/a96524/c21cnsis.htm#2570).

- **HSQLDB**

It supports different concurrency control mechanisms (<https://hsqldb.org/doc/2.0/guide/sessions-chapt.html>). By default, two-phase lock (2PL), but also and multiversion currency control (MVCC).

In both cases, the standard indicates that the database engine returns a higher level of isolation than the requested one in case the requested one is not implemented. Therefore, HSQLDB promotes READ UNCOMMITTED to READ COMMITTED and REPEATABLE READ to SERIALIZABLE.

Aims

- Understand and strengthen the concepts of transaction, concurrence and isolation.
- To know the possibilities of configuring a DBMS depending on the degree of concurrency desired.
- Know the different degrees of isolation of Oracle and HSQLDB.
- To achieve this, the following cases must be tested in the two DBMS indicated and with two levels of isolation:
 - **HSQL (LOCKS) con READ COMMITTED and SERIALIZABLE**
 - **HSQL (MVCC) con READ COMMITTED and SERIALIZABLE**
 - **Oracle con READ COMMITTED and SERIALIZABLE**



Environment

Download the database "Database (hsqldb) for Travel Agency" from the Virtual Campus. Boot it in server mode. Open two sessions with the database and disable Autocommit mode on both.

Boot SQL Developer and connect to Oracle with your username and password. Create the following table (a script is available)

London	450
Paris	320
Rome	280

Open two work sessions with autocommit mode disabled.

Instructions for Setting the Concurrency Control Mechanism in HSQLDB

SET DATABASE TRANSACTION CONTROL LOCKS

SET DATABASE TRANSACTION CONTROL MVCC

Instructions for changing the insulation level

To switch between different levels of isolation, the SQL statement is

SET TRANSACTION ISOLATION LEVEL XXXX

Remember to execute commit or rollback first. If you need to know the current isolation level in HSQLDB you can run

CALL ISOLATION_LEVEL()

Transactions to be executed

For each of the following scripts, it notes the results of each read operation. Are any processes crashing during execution? Which one? When? Why? What anomalies occur in each run?

Important: Make sure to rewrite the original values to the tables before each run.

A. Dirty read

A.1 Script 1

TRANSACTION 1 (Ej: Travel agency)	TRANSACTION 2 (Ej: Booking center)
SET TRANSACTION ISOLATION LEVEL XXXXXXXX	SET TRANSACTION ISOLATION LEVEL XXXXXXXX
SELECT * from Ttrips	UPDATE Ttrips SET price = price + 10

A.2 Script 2

TRANSACTION 1 (Ej: Travel agency)	TRANSACTION 2 (Ej: Booking center)
SET TRANSACTION ISOLATION LEVEL XXXXXXXX	SET TRANSACTION ISOLATION LEVEL XXXXXXXX
SELECT * from Ttrips	UPDATE Ttrips SET price = price + 10
SELECT * from Ttrips	



B. Unrepeatable read

B.1 Script 1

TRANSACTION 1 (Ej: Travel agency)	TRANSACTION 2 (Ej: Booking center)
SET TRANSACTION ISOLATION LEVEL XXXXXXXX SELECT price FROM Ttrips WHERE destination = 'Paris'	SET TRANSACTION ISOLATION LEVEL XXXXXXXX UPDATE Ttrips SET price = 350 WHERE destination = 'Paris'
SELECT price FROM Ttrips WHERE destination = 'Paris'	Commit
Commit	
SELECT price FROM Ttrips WHERE destination = 'Paris'	

B.2 Script 2

TRANSACTION 1 (Ej: Travel agency)	TRANSACTION 2 (Ej: Booking center)
SET TRANSACTION ISOLATION LEVEL XXXXXXXX SELECT * FROM Ttrips WHERE destination = 'Rome'	SET TRANSACTION ISOLATION LEVEL XXXXXXXX UPDATE Ttrips SET price = 500 WHERE destination = 'Rome'
SELECT * FROM Ttrips WHERE destination = 'Rome';	Commit
UPDATE Ttrips SET price=price+10	
SELECT * FROM Ttrips	

Phantom read

TRANSACTION 1 (Ej: Travel agency)	TRANSACTION 2 (Ej: Booking center)
SET TRANSACTION ISOLATION LEVEL XXXXXXXX SELECT * FROM Ttrips WHERE price BETWEEN 250 AND 350	SET TRANSACTION ISOLATION LEVEL XXXXXXXX INSERT INTO Ttrips VALUES ('Madrid',260)
SELECT * FROM Ttrips WHERE price BETWEEN 250 AND 350	Commit